When Runtime and Offline Replay Disagree: A Pinned SABench Audit

Ezekiel Ologunde

ABSTRACT

A deterministic oracle can produce different classifications when its runtime and standalone replay receive different representations of the same action. We audit one released SABench code revision and a separately pinned task revision. Direct replay of all 2,211 published traces yields 346 unauthorized-write cases, compared with 337 explicitly reported in the paper. Reusing the released runtime's thread-to-message normalization changes thirteen calls and nine trace classifications, producing 337 unauthorized-write, 169 external-write and 252 goal-plus-violation cases. All nine classification changes belong to one task template. The documented command-line verifier agrees with direct replay on these nine cases and three unchanged controls. A fresh offline package reproduces all results. This is a bounded artifact correspondence finding, not a new authorization defense, independent validation of user intent, or proof of the paper's actual scoring pipeline.

KEYWORDS

agent evaluation, authorization, replay, normalization, reproducibility

1 SCOPE AND RESEARCH QUESTIONS

SABench separates an agent's task completion from writes outside an explicit authorization contract [1]. It provides synthetic email, calendar and ticket tasks, published traces, a local deterministic oracle and a model runner [2, 3]. Its contract is an evaluation policy, rather than a direct measure of workplace harm. We examine agreement between two released paths that interpret those traces.

RQ1: Does direct replay agree with replay that applies the runtime's existing argument normalization? RQ2: Does the documented standalone verifier reproduce the direct-replay classifications? RQ3: Does the normalization account for the main discrepancy with reported aggregate counts?

The contribution is revision-specific evidence of normalization placement affecting replay, with full trace coverage and a portable supplement. Normalization is already implemented upstream; neither its mechanism nor preprocessing consistency in general is claimed as novel. No exhaustive priority claim is established. This preliminary technical report was prepared on 12 November 2026 and has not undergone independent review.

2 PINNED ARTIFACTS AND METHOD

We pin `hjj194/sabench-artifact` at `1baf318fbdc5`… and the task dataset at `21b40adec3f4`…; full revisions are preserved in the manifest. All 2,222 GitHub blobs, totaling 1,511,229 bytes, match the advertised sizes and Git blob identities and have additional SHA-256 receipts. Three Hugging Face files total 318,527 bytes. The task corpus contains 201 unique IDs, 47 templates and 45 ambiguous tasks. All eleven model-labelled folders cover every task exactly once, with matching template IDs. Folder labels do not independently authenticate model provenance.

We inspect the local core before execution and import it without installing the hosted-model runner. The initial path constructs a `Trace` and calls `oracle_verify`. Per-trace reports and complete aggregates are retained. The study is exploratory: initial outcomes and source inspection preceded the separately frozen normalization protocol.

The paired path selects four reviewed pure functions from the unchanged runtime source using its Python abstract syntax tree: `normalize_write_args` and three helpers. It does not execute the API runner. We copy each task's state, process calls in order, normalize arguments against the evolving copy and use the existing one-operation interpreter to update that copy. Explicit IDs, success flags, steps, messages and final responses are preserved. The adapted trace is then scored by the unchanged oracle.

Six controls check explicit-ID precedence, unread-message resolution, final-message resolution for an all-read thread, an unresolved missing thread, unchanged unrelated-tool arguments and non-mutation of normalization inputs. Before and after comparison, we verify source, task and initial-report hashes. There is no new model inference or external tool action.

3 MECHANISM AND MEASURED RESULTS

The runtime's `execute_tool_call` normalizes an email reply before constructing its `APICall`. If the arguments supply a thread ID without an explicit message ID, the normalizer resolves it to the last unread message, or otherwise the thread's final message. Direct loading of a released trace does not apply that state-dependent step; registry extraction can therefore retain the thread ID as the write target.

For example, task `D-MT-2-001` asks for a reply to Bob about Q3 Plan. Its synthetic thread contains an older read email and a newer unread email. The task authorizes the newer message ID. A released thread-only reply is classified differently when that ID is resolved using the existing runtime convention. This example demonstrates a contract-level correspondence difference, not a general rule that a thread always identifies an authorized message.

All six controls pass. Thirteen calls in thirteen traces receive normalized arguments; nine traces change their goal, CD or external-CD flags. All nine change from goal-false/CD-true/external-true to goal-true/CD-false/external-false. They occur in the D-MT-2 template: four DeepSeek V3.2-labelled traces, two GLM-5-labelled traces and three MiniMax M2.5-labelled traces. Four other argument changes leave these flags unchanged.

The normalized CD, external-CD and goal-plus-CD counts match the paper's explicit Appendix A.3 counts [1]; its goal rate of 89.2% is also recovered by rounding. The conditional goal-plus-CD rate alone is insufficient to diagnose the mismatch: both 252/1,963

Table 1: Full released-trace recount. Normalized replay reuses upstream logic; no new agent run occurs.

Endpoint	Direct	Normalized
Traces	2,211	2,211
Goal satisfied	1,963	1,972
Any unauthorized write (CD)	346	337
External CD	178	169
Goal and CD	252	252

and 252/1,972 round to 12.8%. A separate 0.1-point GPT-5.4-mini goal-display difference remains outside this explanation.

The public CLI, `scripts/oracle_verify.py`, produces full reports identical to the direct path on all nine changed cases and three fixed unchanged controls. Thus the initial result is not solely a peculiarity of our batch import wrapper.

4 INTERPRETATION AND LIMITATIONS

Observed revision history does not explain the main difference. The two public code commits differ only in licence, README and package metadata; the camera-ready task file is byte-identical to our pinned task file. Nevertheless, we have not obtained the paper-production per-trace reports or demonstrated its exact scoring pipeline. Recovering aggregates under an existing normalization path supports correspondence, not proof of author intent or execution history.

This audit does not invalidate SABench's distinction between success and authorization compliance. It explains why direct replay and a runtime-consistent path can differ. The nine affected classifications are dependent observations from one template family, not independent samples of enterprise systems. We neither adjudicate the semantic correctness of authorization labels nor treat model folder names as verified identities. Synthetic external-write labels are not measurements of realized harm.

The experiment is source-informed and post hoc. Six controls and complete trace coverage do not establish universal equivalence of the two execution paths. Other normalization, temporal, multiplicity and tool-semantics boundaries were not exhaustively tested. The observation motivates explicitly recording which representation is replayed; no new security mechanism or demonstrated deployment improvement follows from it.

5 REPRODUCTION AND RESEARCH INTEGRITY

A 1,452,576-byte local supplement bundles the pinned code, synthetic traces and task files under separate upstream notices. Its SHA-256 begins `272cf5ccc68a6506`; the full digest is in the package receipt. A guarded fresh extraction passes four commands: direct replay, normalization comparison, documented CLI checks and full receipt/input verification. All three reference result receipts match, and all 2,233 packaged input hashes remain unchanged. Initial reports are preserved separately from adapted reports.

The supplement requires CPython 3.12 and its standard library. The hosted-agent runner is not used. Code retains upstream MIT terms; traces and tasks retain their declared CC BY 4.0 terms. No general reuse licence has been assigned to our authored scripts or report. Independent human review, maintainer adjudication and external preprint submission have not occurred. AI assistance supported code and manuscript preparation; numerical statements are tied to preserved executable results. No affiliation, funding, venue acceptance or reviewer outcome is invented.

REFERENCES

[1] Jiaju Huang, Shaobin Chen, Xinglong Liang, Xinyu Ma, Hao Yang, Wei Ke, Tao Tan, and Yue Sun. 2026. Task Success Is Not Enough: Side-effect-Aware Evaluation of Tool-Using Language Model Agents. arXiv:2610.34167v1. https://arxiv.org/abs/2610.34167.

[2] SABench authors. 2026. SABench code and evaluation traces. Revision `1baf318fbdc55df2d190467a1b6bf91dfc5f6524`. https://github.com/hjj194/sabench-artifact.

[3] Phaedrus. 2026. SABench task corpus. Revision `21b40adec3f49d954c524c5c5aa95f30c2cb91fd`. https://huggingface.co/datasets/Phaedrus/sabench.